

Практичне завдання 1: Лінійна регресія, оптимізація та статистичний аналіз

Мета завдання

Зрозуміти архітектуру лінійної регресії, навчитися самостійно реалізовувати алгоритми оптимізації, обирати інформативні ознаки, обчислювати метрики якості та перевіряти статистичні припущення моделі.

1 Рекомендований датасет

Для виконання завдання використовуємо **Ames Housing Dataset** (або його спрощену версію з Kaggle: *House Prices - Advanced Regression Techniques*).

- **Цільова змінна:** `SalePrice` (числова, регресія).
- **Числові ознаки:** `LotArea`, `GrLivArea`, `YearBuilt` тощо.
- **Категоріальні ознаки:** `Neighborhood`, `BldgType`, `HouseStyle`, `KitchenQual` тощо.

2 Етапи виконання

2.1 Етап 1. Дослідження даних (EDA), відбір ознак та попередня обробка

1. **Первинний аналіз даних:** дослідіть структуру датасету, типи ознак, пропущені значення, статистичні характеристики та розподіли основних змінних.
2. **Відбір ознак:** визначте, які ознаки доцільно використовувати для побудови моделі.

Проведіть щонайменше базовий аналіз кореляцій між числовими ознаками та цільовою змінною `SalePrice`. Зверніть увагу також на сильні кореляції між самими ознаками, оскільки вони можуть свідчити про мультиколінеарність.

За бажанням можна використати спеціалізовані методи та інструменти відбору ознак, наприклад:

- `SelectKBest` / статистичні тести;
- Recursive Feature Elimination (RFE);
- взаємну інформацію (Mutual Information);
- Lasso як метод відбору ознак;
- інші обґрунтовані методи.

Необхідно пояснити, чому обрано саме ці ознаки. Не обов'язково використовувати всі доступні ознаки датасету.

Важливо: відбір ознак повинен виконуватися без використання тестової вибірки. Якщо метод відбору ознак використовує статистики або параметри, вони повинні оцінюватися лише на Train.

3. **Розбиття вибірки:** розбийте дані на навчальну (Train), валідаційну (Validation) та тестову (Test) вибірки.

Питання: Чому ми повинні розбити дані **до**, а не після масштабування, кодування або інших операцій, які навчаються на даних? Поясніть термін *Data Leakage*.

4. **Кодування категорій:** закодуйте категоріальні ознаки. Обґрунтуйте вибір між One-Hot Encoding, Ordinal Encoding чи Target Encoding.

Питання: Як ви будете обробляти категорії, які з'являться на валідації/тесті, але яких не було в Train?

5. **Масштабування:** застосуйте стандартизацію (або нормалізацію) для числових ознак.

Параметри масштабування повинні бути обчислені лише на Train і потім застосовані до Validation та Test.

2.2 Етап 2. Реалізація Лінійної Регресії засобами numpy

Реалізуйте клас або набір функцій лінійної регресії **виключно використовуючи numpy**.

Модель повинна мати вигляд:

$$\hat{y} = \mathbf{X}\mathbf{w} + b.$$

Реалізуйте:

- обчислення $\hat{y}$;
- функції втрат (наприклад, Mean Squared Error);
- обчислення градієнтів за параметрами $\mathbf{w}$ та b ;
- оновлення параметрів моделі.

Використання циклів `for` для обчислення Cost Function або градієнтів **заборонено** — використовуйте векторизацію та матричні операції `numpy`.

Normal Equation реалізовувати не потрібно. Як бонус, можна навести його виведення та проаналізувати його обмеження. +3 додаткові бали.

Навчання моделі на наступних етапах повинно виконуватися виключно за допомогою градієнтних методів.

2.3 Етап 3. Градієнтний спуск: три варіанти оптимізації

Реалізуйте та навчіть модель, використовуючи три варіанти градієнтного спуску (з нуля, на `numpy`):

1. **Batch GD** (повний градієнтний спуск);
2. **SGD** (стохастичний, по одному прикладу);
3. **Mini-batch GD** (розмір батчу B підберіть самостійно, наприклад, 32 або 64).

Для кожного методу необхідно:

- реалізувати алгоритм оновлення ваг;
- підібрати Learning Rate (α);
- зберігати значення функції втрат протягом навчання;
- визначити критерій зупинки або максимальну кількість ітерацій.

Завдання: побудуйте графік залежності Cost Function від кількості ітерацій (Epochs) для всіх трьох методів на одному рисунку. Проаналізуйте поведінку графіків.

Поясніть:

- який метод збігається найстабільніше;
- який метод має найбільші коливання Cost Function;
- як розмір batch впливає на швидкість та стабільність навчання;
- як вибір Learning Rate впливає на збіжність.

2.4 Етап 4. Оцінка якості та коефіцієнт детермінації (R^2)

Обчисліть метрику R^2 для навчальної, валідаційної та тестової вибірок:

$$R^2 = 1 - \frac{\sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2}{\sum_{i=1}^m (y^{(i)} - \bar{y})^2}.$$

- Поясніть статистичний зміст R^2 .
- Чи може R^2 бути від'ємним? Якщо так, то за яких умов?
- Порівняйте R^2 на Train та Test. Що це говорить про стан моделі?
- Порівняйте якість трьох варіантів градієнтного спуску.

2.5 Етап 5. Перевірка статистичних припущень

Лінійна регресія — це статистична модель. Ви повинні перевірити та візуалізувати наступні припущення на основі **залишків (residuals, $\epsilon = y - \hat{y}$)**:

1. **Лінійність (Linearity):** графік *Residuals vs Fitted values*.
2. **Нормальність залишків (Normality):** Q-Q plot, тест Шапіро-Уїлка або гістограма залишків.
3. **Гомоскедастичність (Homoscedasticity):** графік *Scale-Location*. Чи є дисперсія помилок однаковою?
4. **Відсутність мультиколінеарності:** матриця кореляції або VIF (Variance Inflation Factor).

Вимога до звіту: не просто наведіть графіки, а напишіть висновок:

«Припущення X виконується/порушується, тому що... Це означає, що...».

Окремо поясніть, чи виявлена мультиколінеарність вплинула на стабільність або інтерпретацію отриманих коефіцієнтів моделі.

2.6 Етап 6. Завдання для глибокого розуміння

Виконайте **один** з трьох наведених нижче експериментів.

- **Експеримент А (Вплив масштабування):** запустіть Mini-batch GD на даних без стандартизації, а потім зі стандартизацією. Порівняйте швидкість збіжності та підберіть Learning Rate (α) для обох випадків. Поясніть геометрично, чому без масштабування GD працює гірше.
- **Експеримент Б (Learning Rate):** для кожного з 3 варіантів GD знайдіть таке α , при якому алгоритм **розходиться** (Cost прямує до нескінченності або стає нестабільним). Порівняйте поведінку трьох методів та поясніть, чому їхня чутливість до Learning Rate відрізняється.
- **Експеримент В (Мультиколінеарність):** якщо на Етапі 5 ви знайшли суттєву мультиколінеарність, реалізуйте **Ridge Regression (L2-регуляризацію)** засобами `numpy`. Додайте штраф

$$\frac{\lambda}{2m} \|\mathbf{w}\|^2$$

до Cost Function та виведіть нову формулу для градієнта. Покажіть, як змінилися ваги ($\mathbf{w}$) порівняно зі звичайною регресією.

Формат здачі

Jupyter Notebook (`.ipynb`), завантажений на Github репозиторій, який містить:

- чистий, прокоментований код;
- Markdown-комірки з теоретичними поясненнями та висновками;
- результати EDA та обґрунтування вибору ознак;
- графіки навчання та статистичного аналізу;
- відповіді на теоретичні питання;
- порівняння трьох варіантів градієнтного спуску.

Код повинен бути структурований. Усі алгоритми градієнтного спуску та обчислення градієнтів для моделі лінійної регресії повинні бути реалізовані самостійно засобами `numpy`.